INFORME DE PROYECTO RUTAS AEREAS

1. Descripción de estructuras usadas y su razón

CargadorGrafo lee los aeropuertos y las rutas para construir un *Grafo* dirigido con listas de adyacencia. Un *HashMap* relaciona cada código con su *Aeropuerto* y otro, con una *ArrayList* de salidas. Esto permite localizar los datos por código y recorrer las conexiones existentes. Cada *Ruta* guarda destino, minutos y costo; el origen corresponde a la lista que la contiene. La conexión inversa requiere otro registro.

rutasOriginales conserva listas distintas de las activas, pero comparte con ellas los objetos *Ruta*, cuyos valores permanecen fijos. Cancelar retira todas las conexiones directas del origen al destino indicado. Restaurar repone las referencias desde ese respaldo, sin modificar los archivos.

BuscadorRutas consulta el grafo activo. BFS utiliza *ArrayDeque* como cola FIFO para explorar por cantidad de vuelos. DFS la usa como pila para continuar desde el último aeropuerto pendiente. Ambos registran vértices en un *HashSet*. Dijkstra emplea una *PriorityQueue* para extraer la propuesta de menor peso acumulado y un *HashMap* de mejores valores. *CriterioPeso* selecciona minutos o costo en el mismo algoritmo.

BFS y Dijkstra guardan el predecesor y la ruta exacta de llegada en mapas. Reconstruyen desde el destino e invierten los tramos. *ResultadoRecorrido* conserva códigos y tramos en listas de solo lectura, y minutos y costo totales en campos separados.

PrincipalController coordina consultas y modificaciones. Entrega el resultado a *MapaGrafo*, que dibuja conexiones y estados de la red, y a *AnimadorVuelo*. Este prepara las transiciones y pausas entre vuelos e interpola la posición y los acumulados durante la reproducción.

2. Complejidad de operaciones

Se consideran campos de tamaño acotado, consultas de costo esperado constante en *HashMap* y *HashSet*, e inserciones al final de *ArrayList* y operaciones de *ArrayDeque* amortizadas. $\log x$ abrevia $\log(\max\{x, 2\})$. n es la cantidad de aeropuertos, m las rutas activas y m_0 las originales, incluidas las canceladas. $d(u)$ y $d_0(u)$ son las salidas activas y originales de u. p es la cantidad de aeropuertos del itinerario, igual a sus vuelos más uno.

Operación	**Costo temporal**
Cargar archivos y construir el grafo	$O(n + m_0)$
Agregar un aeropuerto o una ruta	$O(1)$
Localizar un aeropuerto por código	$O(1)$
Copiar la colección de aeropuertos	$O(n)$
Copiar las salidas de u	$O(d(u))$

Operación	Costo temporal
Cancelar conexiones de u hacia un destino	$O(d(u))$
Restaurar todas las rutas	$O(n + m_0)$
Identificar las rutas canceladas de u	$O(d(u) + d_0(u))$
Contar las rutas activas	$O(n)$
Menos vuelos mediante BFS	$O(n + m)$
Alcanzabilidad mediante DFS	$O(n + m)$
Menor tiempo o costo con Dijkstra	$O((n + m) \log m)$
Reconstruir el itinerario y crear el resultado	$O(p)$
DFS y listas alfabéticas de alcanzables/no alcanzables	$O(n \log n + m)$
Redibujar el mapa completo	$O(n^2 + m_0 + p)$
Preparar o reiniciar la secuencia de vuelo	$O(p)$
Interpolar posición y acumulados del avión	$O(1)$

Restaurar recorre los aeropuertos y repone sus listas desde las originales. Identificar canceladas compara estas con un conjunto de activas. Contar rutas suma los tamaños de las listas. Las copias de salidas de las búsquedas forman parte del trabajo $O(n + m)$.

BFS marca al encolar y DFS al extraer. DFS descarta los pendientes repetidos antes de expandirlos: su pila ocupa $O(n + m)$ como cota superior y sus visitados, $O(n)$. Dijkstra añade una entrada por mejora y descarta propuestas obsoletas al extraerlas. La cola ocupa $O(m + 1)$, incluido el origen. Con un grafo simple, su costo también puede expresarse como $O((n + m) \log n)$.

Los predecesores forman un árbol implícito: $p \leq n$, y reconstruir queda incluido en las búsquedas. Los getters de *ResultadoRecorrido* cuestan $O(1)$, pues devuelven las referencias de sus listas o los totales almacenados. La presentación de alcanzabilidad agrega el ordenamiento alfabético a DFS.

El dibujo recorre conexiones e itinerario y compara cada candidata de etiqueta con hasta $O(n)$ espacios ocupados, produciendo el término $O(n^2)$. Preparar la animación requiere $O(p)$ tiempo y memoria. Cada actualización interpola en $O(1)$, pero solicita el dibujo completo. Las acciones de la interfaz suman ese costo al del modelo; consultar un aeropuerto incluye además preparar sus salidas en $O(d(u))$.

3. Respuestas a las preguntas de análisis

1. Comparación de los recorridos.

Los siguientes resultados corresponden a los archivos asignados, con las 202 rutas activas. BFS obtiene el recorrido con menos vuelos y Dijkstra calcula los de menor tiempo y costo.

Viaje	Criterio	Recorrido	Vuelos	Minutos	Costo ($)
DEN → SJO	Menos vuelos	DEN → LAX → SJO	2	500	895
DEN → SJO	Menor tiempo	DEN → DFW → IAH → SJO	3	385	525
DEN → SJO	Menor costo	DEN → DFW → SAL → MGA → SJO	4	425	435
LIR → SJU	Menos vuelos	LIR → JFK → SJU	2	510	585
LIR → SJU	Menor tiempo	LIR → SJO → PTY → CCS → SJU	4	355	520
LIR → SJU	Menor costo	LIR → SAL → MIA → SJU	3	385	405
SJO → MVD	Menos vuelos	SJO → MIA → BSB → GRU → MVD	4	870	1125
SJO → MVD	Menor tiempo	SJO → PTY → LIM → EZE → MVD	4	595	685
SJO → MVD	Menor costo	SJO → PTY → LIM → GRU → MVD	4	720	640

Tener menos vuelos no garantiza llegar antes, porque cada conexión tiene una duración distinta. DEN → SJO requiere 500 minutos con dos vuelos, mientras que el recorrido de tres vuelos tarda 385. En SJO → MVD los tres resultados tienen cuatro vuelos: BFS puede elegir cualquiera entre los que tienen esa cantidad mínima, aunque sus minutos y costos sean diferentes.

La ruta más rápida tampoco garantiza el menor precio. Para DEN → SJO, el recorrido de 385 minutos cuesta $525 y el más barato cuesta $435, pero tarda 425 minutos. Cada búsqueda minimiza el criterio seleccionado.

2. BFS, Dijkstra y pesos negativos

BFS encuentra la menor cantidad de vuelos porque explora por niveles de conexiones. Cada vuelo cuenta como una arista, independientemente de sus minutos. Por eso puede encontrar antes un recorrido con menos vuelos, aunque tarde más que otro con escalas adicionales.

Dijkstra requiere que los pesos utilizados sean no negativos. Con esa condición, el menor valor acumulado vigente que se extrae de la cola ya es definitivo. En el proyecto, Grafo rechaza minutos o costos negativos al agregar una ruta y permite valores de cero.

Si se admitiera un costo negativo, Dijkstra perdería esa garantía: una conexión posterior podría reducir el costo de un aeropuerto ya procesado. Habría que utilizar un algoritmo que admita pesos negativos. Además, si un ciclo de costo negativo es alcanzable desde el origen y permite continuar hasta el destino, repetirlo reduciría el costo sin límite y no existiría un mínimo finito.

3. Alcanzabilidad desde y hacia SJO

Con las 202 rutas originales activas, KIN es el único aeropuerto inalcanzable desde SJO. Los otros 38 destinos son alcanzables, sin contar el origen. Desde ASU y MVD no es posible llegar a SJO. Como el grafo es dirigido, llegar de un aeropuerto a otro no garantiza que exista un camino de regreso.

BuscadorRutas.buscarAlcanzables realiza DFS desde el origen mediante una pila. Registra los aeropuertos visitados y recorre sus conexiones salientes activas. La interfaz compara ese conjunto con todos los aeropuertos para separar los destinos alcanzables de los no alcanzables y excluye al origen de ambas listas.

Para comprobar desde dónde se llega a SJO, una verificación auxiliar ejecutó el mismo método desde cada aeropuerto y revisó si SJO pertenecía al conjunto obtenido. Así se identificaron ASU y MVD.

4. Cancelación que deja un destino inalcanzable

Cancelar LIM → CUZ deja a CUZ inalcanzable desde SJO. Los destinos alcanzables pasan de 38 a 37. KIN ya era inalcanzable antes de la cancelación.

Una verificación auxiliar probó por separado las 202 conexiones. Primero obtuvo con DFS los aeropuertos alcanzables desde SJO. Después canceló cada conexión, repitió la consulta y comparó el resultado con el conjunto inicial. Restauró todas las rutas antes del siguiente intento.

LIM → CUZ fue la única cancelación individual que hizo perder un destino alcanzable desde SJO con los datos asignados. Esta conclusión corresponde a eliminar una conexión a la vez. Al restaurarla, CUZ vuelve a ser alcanzable y el grafo recupera sus 202 rutas activas.

5. Lista de adyacencia y matriz.

La lista de adyacencia permite almacenar las conexiones existentes y recorrer directamente las salidas de cada aeropuerto. Su espacio es $O(n + m)$, mientras que una matriz requiere $O(n^2)$, porque reserva una celda por cada par de aeropuertos, incluso si no existe una conexión.

Red	**Rutas (m)**	**Lista: n + m**	**Matriz: n^2**
40 aeropuertos	202	242	1600
4000 aeropuertos	≈80 000	≈84 000	16 000 000

Para la segunda red, unas 20 salidas por aeropuerto representan aproximadamente $4000 \times 20 = 80\,000$ rutas dirigidas. El número de rutas crece mucho menos que el de pares posibles, por lo que la lista resulta adecuada para esta red dispersa. La comparación cuenta vértices y rutas frente a celdas de matriz; los bytes dependen de las estructuras y los tipos utilizados.

En este proyecto, *rutasOriginales* añade listas de respaldo que comparten los objetos *Ruta* con las activas. Por eso, considerando también las conexiones canceladas que se conservan para restaurar, el espacio del grafo es $O(n + m_0)$, donde m_0 es la cantidad de rutas originales.

6. Complejidad de las búsquedas y cola de prioridad

Algoritmo	**Costo temporal**
BFS	$O(n + m)$
DFS	$O(n + m)$
Dijkstra	$O((n + m) \log m)$

BFS y DFS expanden cada aeropuerto una vez y recorren sus conexiones salientes. Las copias de esas listas forman parte del costo $O(n + m)$. En DFS pueden acumularse pendientes repetidos, pero se descartan antes de expandirlos.

Dijkstra utiliza *PriorityQueue*. Cada mejora agrega una propuesta y las obsoletas se descartan al extraerlas. La cola puede contener $O(m + 1)$ entradas y sus inserciones y extracciones cuestan $O(\log m)$, con la convención de logaritmos indicada en el análisis. Para un grafo simple, la cota también puede expresarse como $O((n + m) \log n)$.

Una versión que busca el mínimo recorriendo los n vértices en cada paso requiere $O(n^2 + m)$: hasta n selecciones lineales y el recorrido de las conexiones. La cola de prioridad mantiene organizadas las propuestas y permite extraer el mínimo sin hacer ese recorrido completo. La mejora resulta especialmente útil en redes dispersas, donde hay muchas menos conexiones que pares posibles de aeropuertos.